RTL-to-C Reverse Engineering for HLS

Dr. Chandan Karfa

Department of Computer Science and Engineering

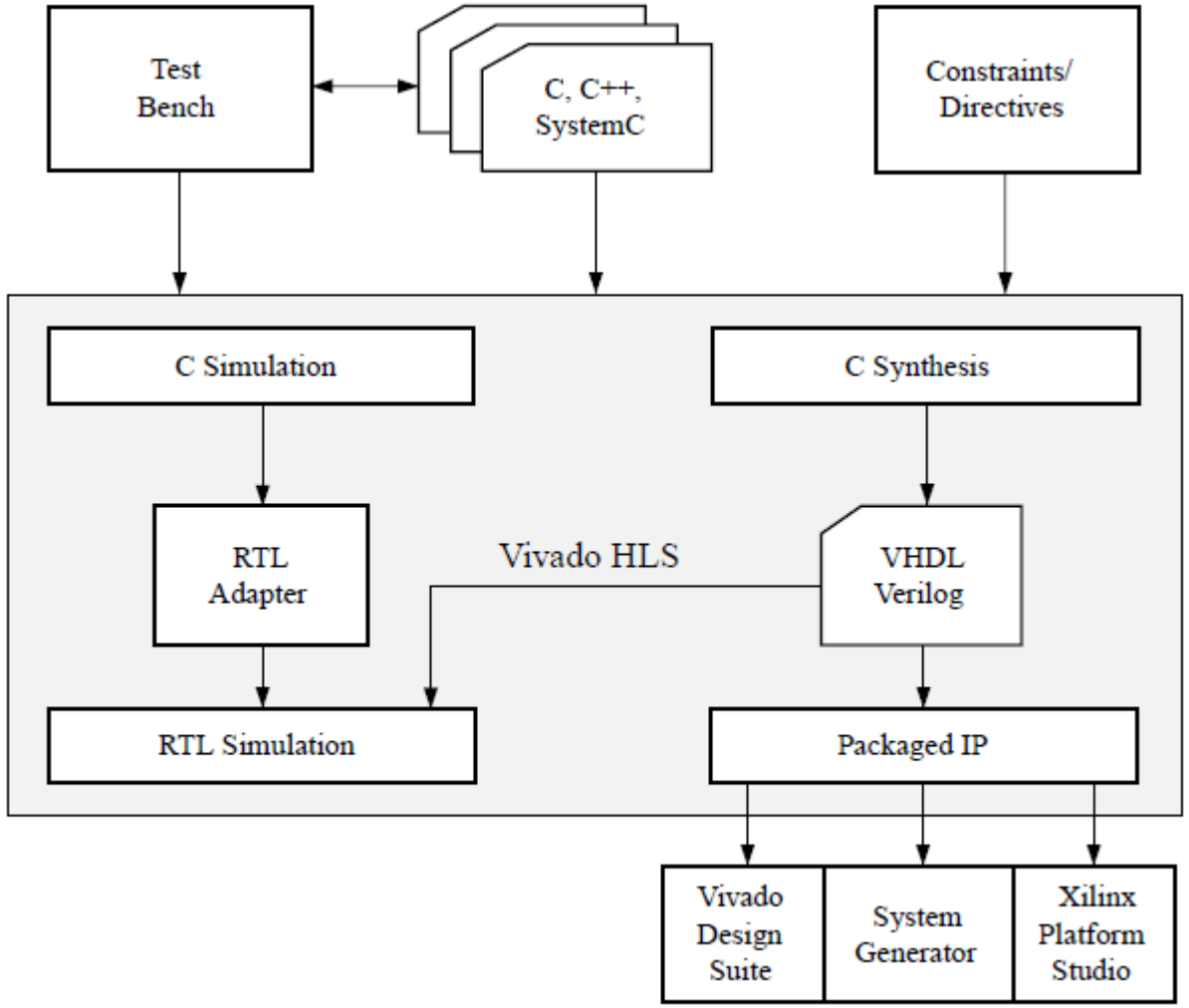


भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Reference

- M. Abderrahman, J. Patidar, J. Oza, Y. Nigam, C. Karfa "*FastSim: A Fast Simulation Framework for High-Level Synthesis*", IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, June 2021.

Simulation based Verification of HLS



Motivation

- RTL Simulation is VERY slow compared to C-simulation

Bench	C-sim(s)	RTL co-Sim(s)	Modelsim(s)
des	28.01	34672	36024
mips	0.985	2620	2885
aes_enc	12.656	4389	4693
aes_dec	14.442	4467	4780

Objective and Problem Statement

Our aim is to perform RTL to C reverse engineering to generate equivalent C-code.

MUST satisfy:

- fast simulation
- functional correctness of the RTL
- cycle accurate simulation
- accurate performance estimation
- generate a highly readable and debug friendly simulation code

MUST Supports:

- array mapping to memory modules,
- non-inlined function calls,
- parallel execution frameworks invoked by loop unrolling, pipelining, task level piplining etc.
- accurate simulation of pipeline stalls

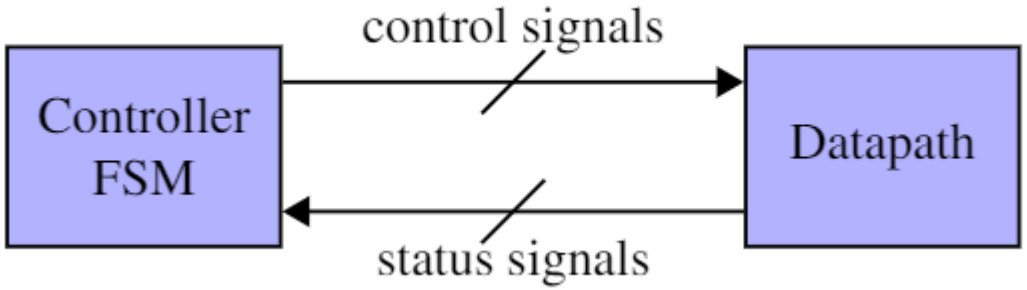
Simulation Based Verification of HLS

- **Verilator** simulator parse the generic Verilog to C++ code
  - Disregards the FSMD structure of the HLS generated RTL due to which impacts both simulation and debugging performance.
- A recent work **FLASH** uses scheduling information to verify the RTL
  - Not able to detect the bug in allocation and binding phase and datapath and controller generation phase.

- Verilator: https://www.veripool.org/
- Y. Choi, Y. Chi, J. Wang, and J. Cong, “FLASH: Fast, parallel, and accurate simulator for HLS,” IEEE TCAD, pp. 1-14, 2020

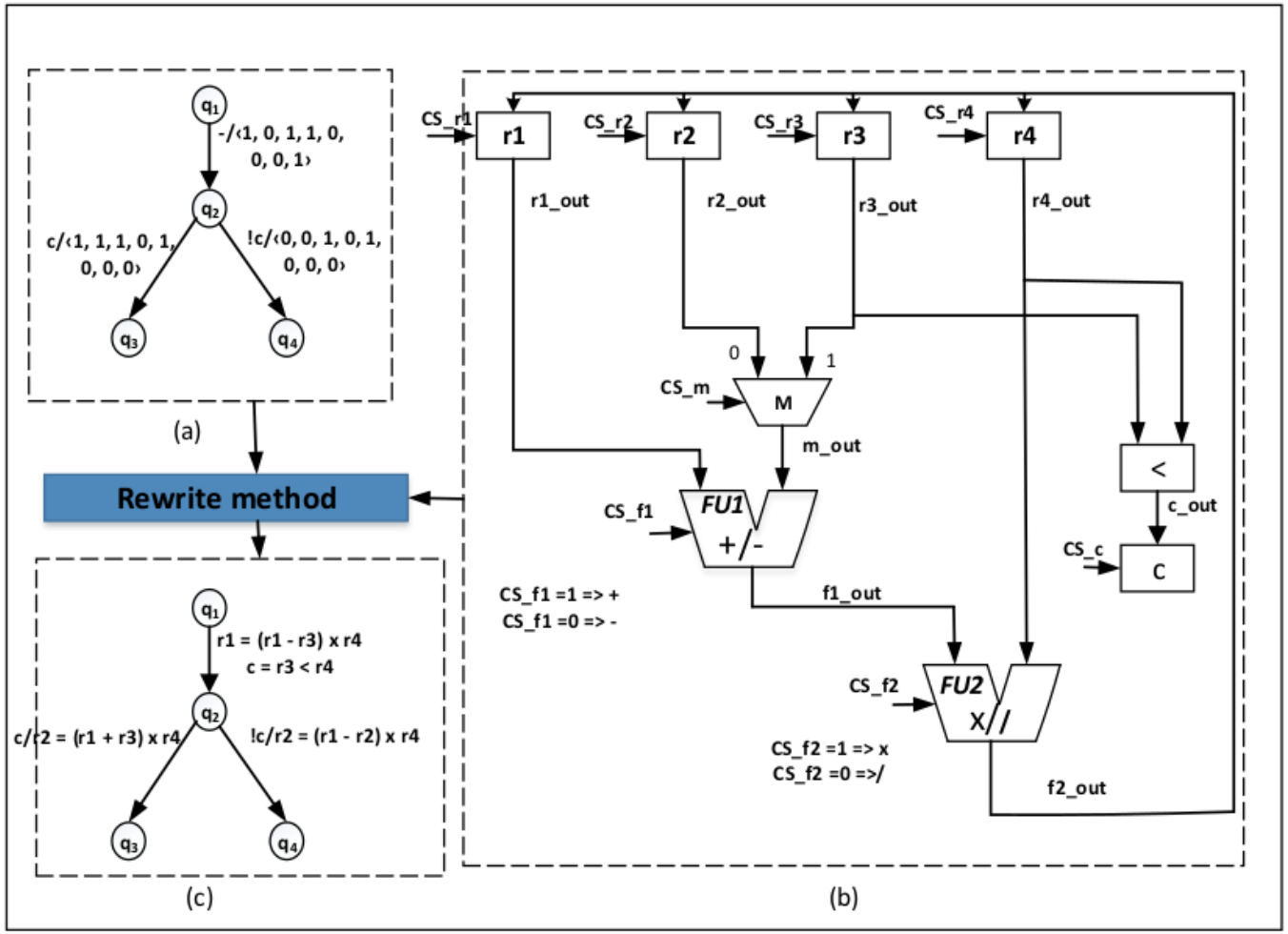
RTL to C Conversion

Idea: Take advantage of this RTL structure during RTL to C conversion



- In each state, the controller assigns 0/1 value to each control signal.
- As a result, a set of RT operations are performed in the datapath.
- The datapath sends results of some conditional checks to the controller. The FSM state transitions depend on those status signals.
- We analyze the datapath using control signals values in the particular state to identify the RT operation(s) executed in the datapath.

RTL to C Conversion



Rewrite Method

- Obtain set of Micro-operations (M) in the Data-path and its corresponding control assignment.
- Obtain active micro-operations M_A (subset of M) for a given control signal assignment A.
- Find the set of register transfer (RT) operations performed by the micro-operations in M_A.
- The spatial sequence of the micro-operations is in the reverse order of the way they are selected by the rewriting method.

Rewrite Method

r1 = fOut

fOut = fLin – fRin

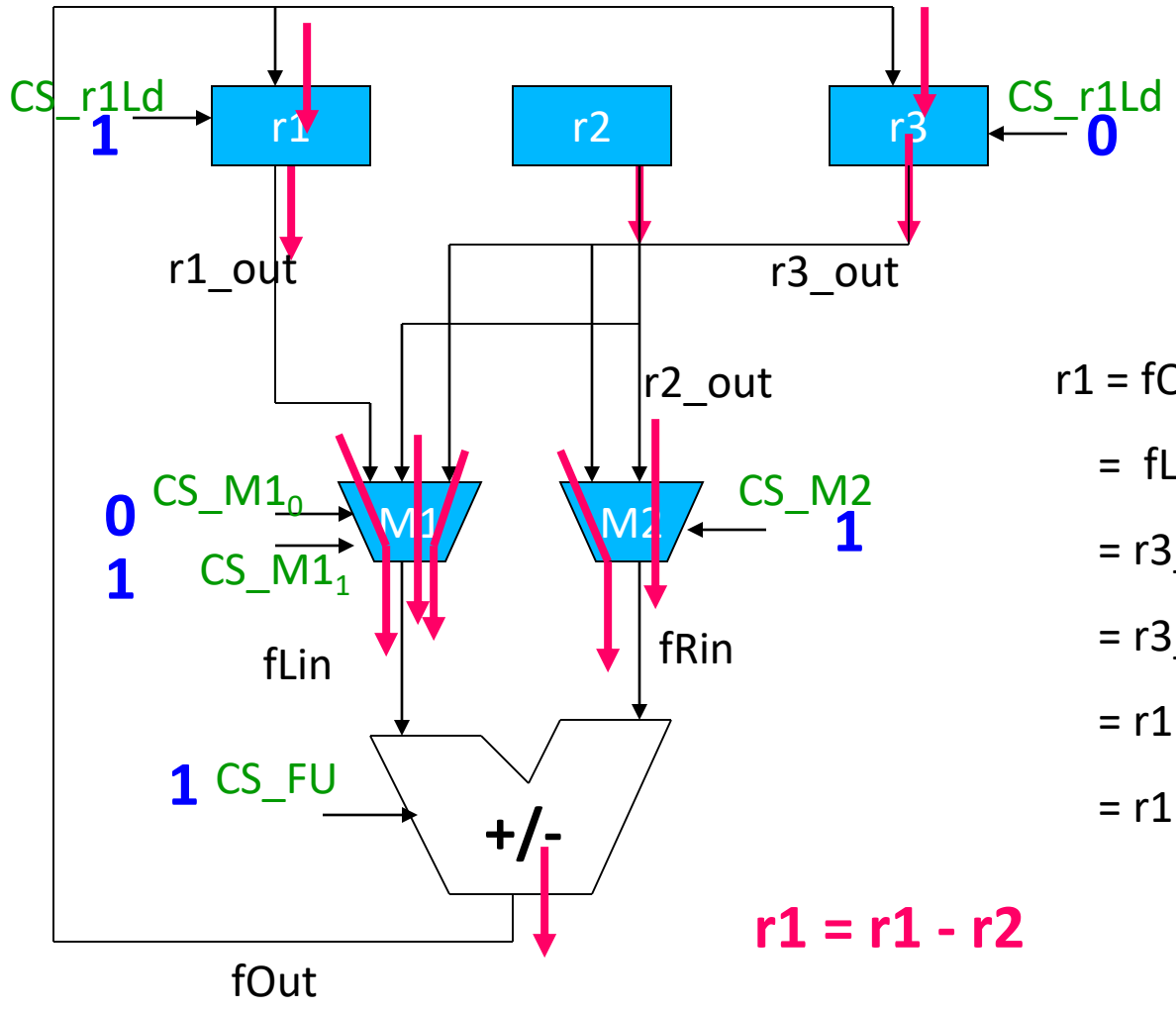
fLin = r3_out

fRin = r2_out

r1_out = r1

r2_out = r2

R3_out = r3



r1 = fOut

= fLin - fRin

= r3_out – fRin

= r3_out – r2_out

= r1 – r2_out

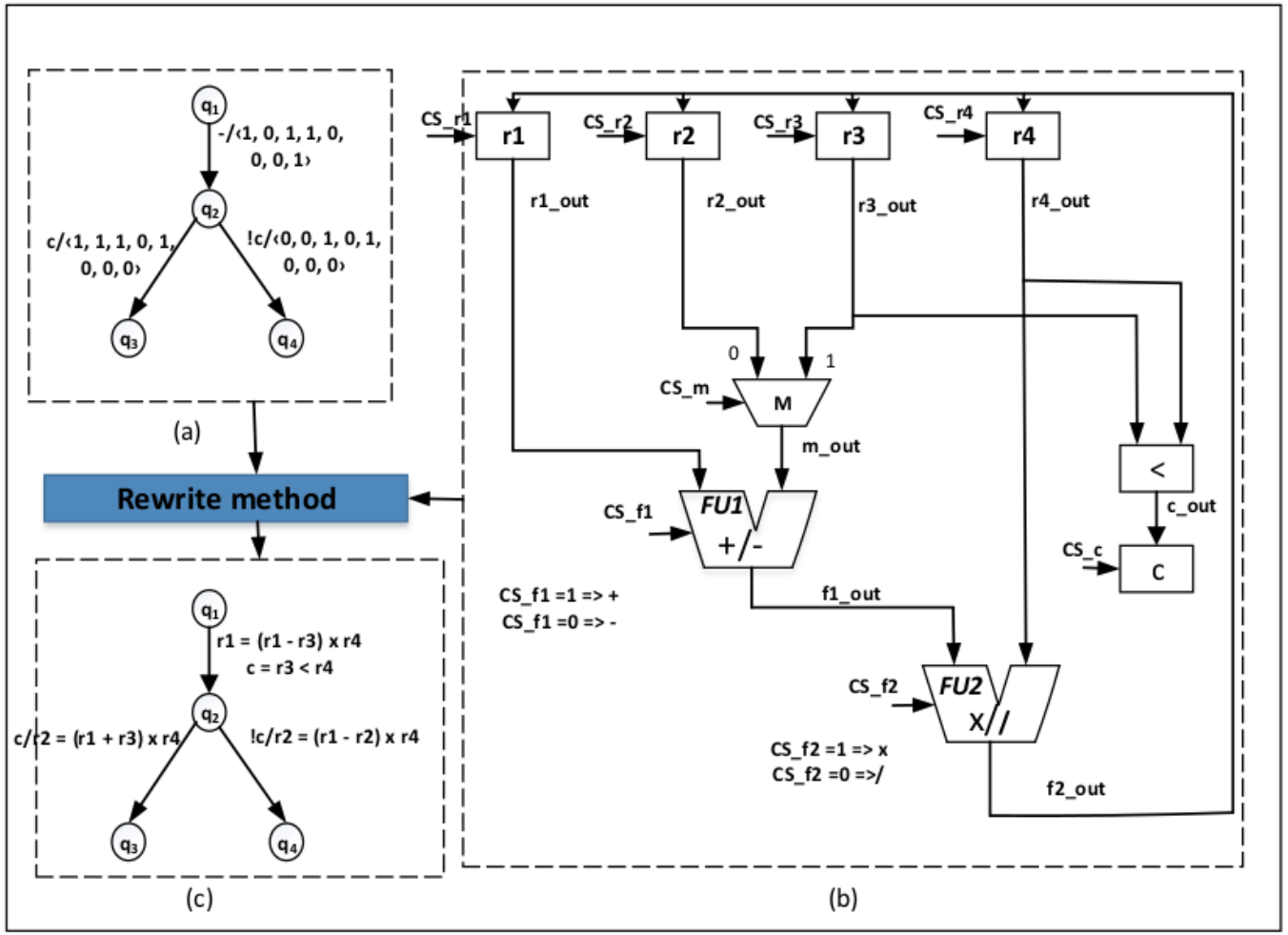
= r1 – r2

Rewrite Method

Algorithm 1: Rewrite (RTL)

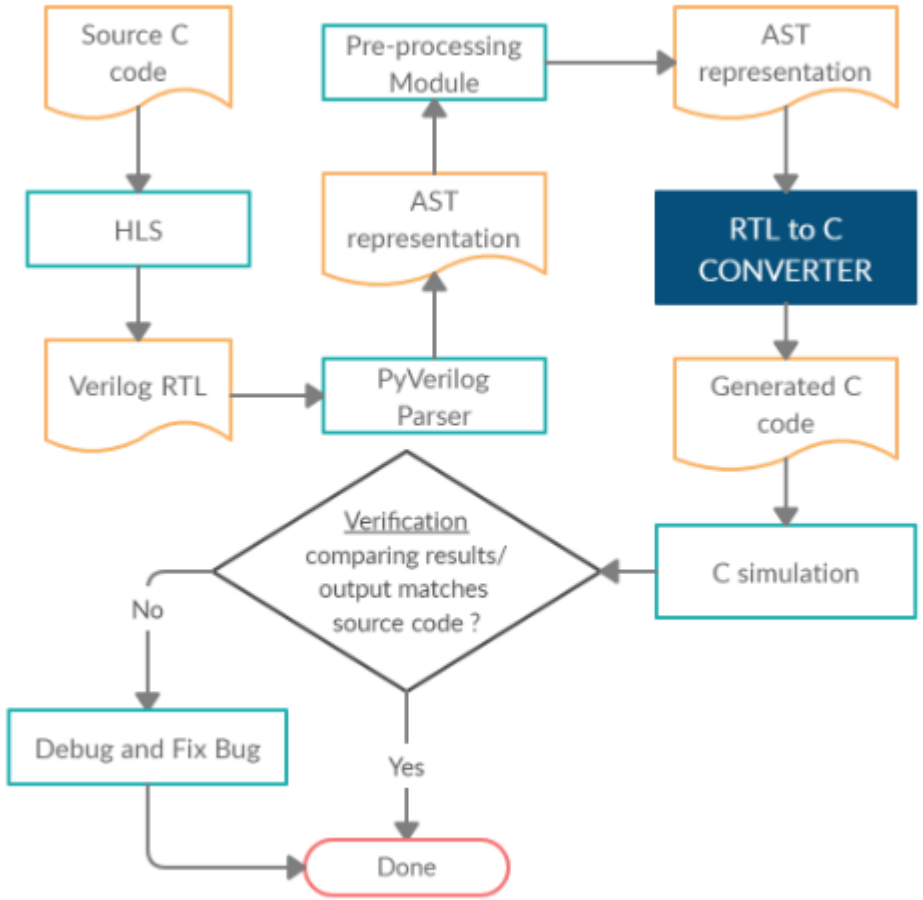
Input: RTL
Result: An FSMD
/* RTL consists of a Data path D and a controller FSM F */
1 **foreach** state S in the controller FSM F **do**
2 Find the active micro-operations M_S for the control signal assignments in S;
3 $R_S = \Phi$; /* Set of RT-operations in S */
4 **foreach** micro-opn of the form $\mu : r \Leftarrow r_{in}$ in M_S **do**
5 **do**
6 w = Find the left-most wire signal in the RHS exp μ_e of μ;
7 Find a micro-opn of the form $w \leftarrow e_w$ in M_S;
8 Replace w with (e_w) in the μ_e;
9 **while** (all signals in RHS exp μ_e of μ are either Input, Reg or Constant);
10 $R_S = R_S \cup \{\mu\}$;
11 **end foreach**
12 Replace the control signal assignments in S of F with R_S;
13 **end foreach**
14 Return F; /* FSM F is converted to FSMD */

Generate C




```
Void main(){
  int r1, r2, r3, r4;
  q1: //state label
       r1 = (r1 – r3) x r4;
      c = r3 < r4;
      go to q2;
  q2:
     if(c){
          r2 = (r1 + r3) x r4;
          go to q3;}
     else{
          r2 = (r1 – r3) x r4;
          go to q4;}
  q3:
    ……
  q4:
    ……
```

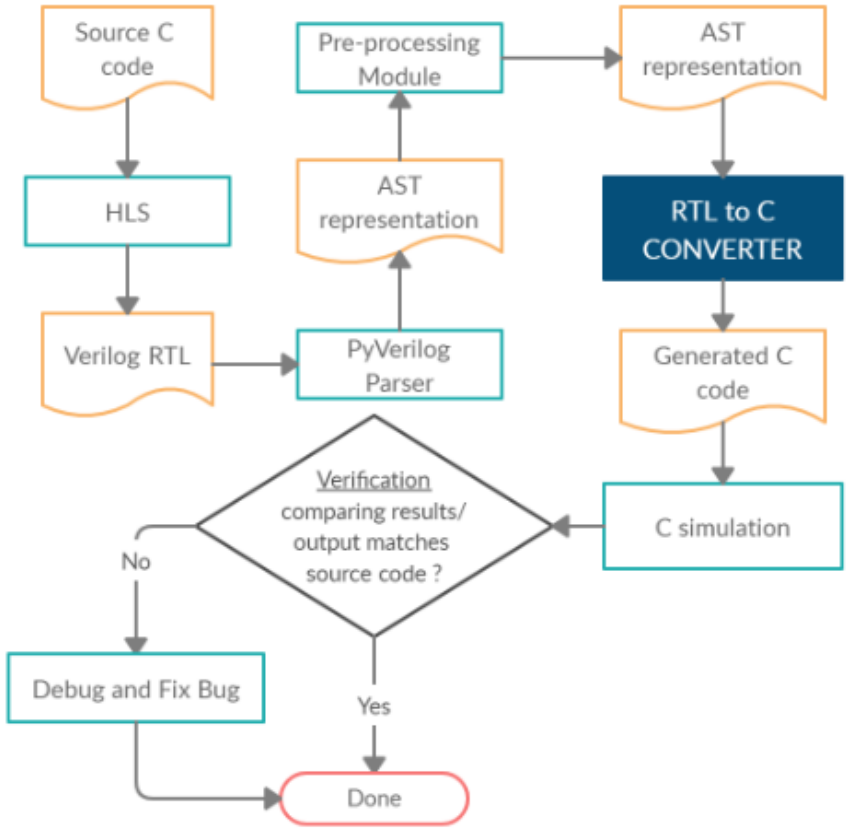
Overall Flow



RTL-To-C Conversion

The main Components of our framework.

- **Pre-processing Verilog**
- AST representation
- Parser implementation



Pre-processing Verilog

Need of pre-processing:

- In Verilog HDL, there are certain operations which perform manipulation at bit level.
- C language doesn't provide such bit level manipulation on variables.
- So, we need to convert such operations into combination of some supported operations like bit-wise AND, bit-wise OR, left shift, right shift etc. to get the actual behaviour.

So far, we have identified two operation that require pre-processing are as follows:

A. Concat Operation

B. Part-select operation

Concat Operation Preprocessing

- It concats the contents of one or more register at the end of another register.
- Multiple concatenations may be performed which is known as replication.

```
// Concat operation:
reg [WIDTH_A - 1 : 0] reg_A;
reg [WIDTH_B - 1 : 0] reg_B;
reg [WIDTH_C - 1 : 0] reg_C;
assign reg_A = {{reg_B}, {reg_C}};

// After pre-processing:
wire [(WIDTH_A - 1):0] temp_0, temp_1;
assign temp_0 = reg_B << width_C;
assign temp_1 = temp_0 | reg_C;
assign reg_A = temp_1;
```

Fig. 1.4

Part-select Operation Preprocessing

- Operation select specific part of the register.

```
// Part-select operation:
reg [63:0] reg_A, reg_B;
always @ (*) begin
    reg_A[40:0] <= reg_B[51:11];
end
// After pre-processing:
wire [63:0] temp_2, temp_3, temp_4;
always @ (*) begin
    reg_A <= temp4;
end
// width of register reg_A = 64 bit.
assign temp_2 = reg_B & 64'd4503599627368448;
assign temp_3 = reg_A & 64'd18446741874686296064;
assign temp_4 = temp_2 | temp_3;
```

Fig. 1.5

AST representation

- An abstract syntax tree(AST) is the syntactic representation of the source code written in a programming language.
- To generate this syntax tree representation, we used Pyverilog library, which converts Verilog code into a tree-like representation.
- In our parser we use this syntax tree representation of Verilog source code generated by Vivado.

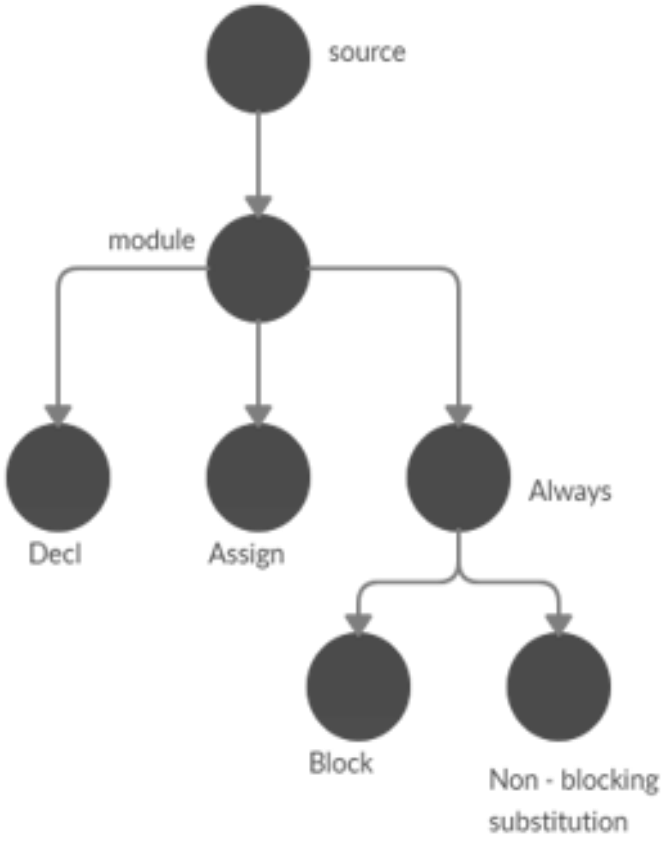
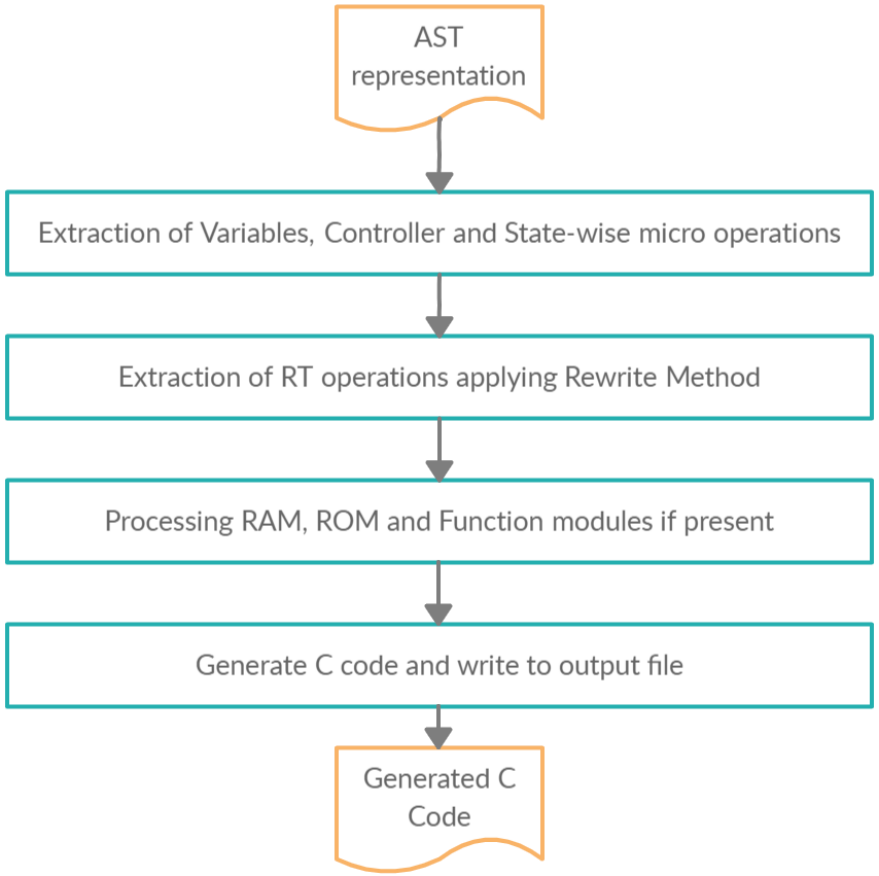


Fig. 1.5

Parser Implementation



RAM, ROM and Function Module

RAM, ROM and Function Module are present in separate verilog file and processing of these module require some information which is taken as user input.

1. No. of RAM and ROM modules.
2. Name of all RAM and ROM modules.
3. No of ports for each RAM and ROM modules.
4. No. of Function modules.
5. Name of all Function modules.
6. No. of RAM and ROM passed as input argument to the module.
7. Instance Name of all input RAM and ROM.
8. No. of port for each Input RAM and ROM.
9. Location of all the Verilog files

RAM and ROM Module

```
shiftingcidi_sub_dEe #(
    .DataWidth( 64 ),
    .AddressRange( 16 ),
    .AddressWidth( 4 ))
sub_key_U(
    .clk(ap_clk),
    .reset(ap_rst),
    .address0(sub_key_address0),
    .ce0(sub_key_ce0),
    .we0(sub_key_we0),
    .d0(sub_key_load_reg_170),
    .q0(sub_key_q0)
);
```

Fig. 2.1 RAM Instance

```
shiftingcidi_itercud #(
    .DataWidth( 2 ),
    .AddressRange( 16 ),
    .AddressWidth( 4 ))
iteration_shift_U(
    .clk(ap_clk),
    .reset(ap_rst),
    .address0(iteration_shift_address0),
    .ce0(iteration_shift_ce0),
    .q0(iteration_shift_q0)
);
```

Fig. 2.2 ROM Instance

RAM and ROM (Continued....)

Parser identify status of signal “ce0” and “we0” in each state and if these signal found set then place the RAM/ROM block at the end of that state in generated C-code.

```
if(1 == ap_CS_fsm_state5){
    sub_key_address0 = sub_key_addr_reg_455__temp ;
    sub_key_ce0 = 1;
    sub_key_we0 = 1;
}
if(sub_key_ce0){
   sub_key_q0=sub_key_ram[sub_key_address0];
    if(sub_key_we0){
       sub_key_ram[sub_key_address0]=sub_key_load_reg_170;
    }
}
goto ap_ST_fsm_state2;
```

Fig. 2.3

Function Module

- Function module mimics the idea of function calls in C program.
- HLS tool usually generate separate verilog file for function module having a datapath and a controller FSM.
- In top module there is instance for the function module as shown in Fig. 2.4

```
calculationofkeys grp_calculationofkeys_fu_193(
    .ap_clk(ap_clk),
    .ap_rst(ap_rst),
    .ap_start(grp_calculationofkeys_fu_193_ap_start),
    .ap_done(grp_calculationofkeys_fu_193_ap_done)
    .L(L),
    .R(R)
);
```

Fig. 2.4

Function Module (Continued...)

- Similar to the RAM/ROM module parser identifies the status of “ap_start” signal of that function, if it is set then place the function call in that state as shown in Fig. 2.5.
- Our parser supports hierarchy of function calls as well. Since, the top module waits until the completion of execution of the module it called, cycle accurate simulation is achieved by following the states of the respective FSMs.

```
if((exitcond2_fu_233_p2 == 1) && (1 == ap_CS_fsm_state2))
{
    grp_calculationofkeys_fu_193_ap_start_reg =  1;
}
 if (grp_calculationofkeys_fu_193_ap_start_reg==1){
     grp_calculationofkeys_fu_193_ap_start=1;
     calculationofkeys(&L,&R,&ap_clk,&grp_calculationofkeys_fu_193_ap_done,
     grp_calculationofkeys_fu_193_ap_done=1;
 }
goto ap_ST_fsm_state8;
```

Fig. 2.5 Sample example of function in generated C-code

Generate C-code

```
#include<stdio.h>
#define CONSTANT
function_prototypes();
int main(){
    //variable declaration;
    //RAM,ROM declaration;
    state1_label:
        //copy old_var_value=new_var_value;
        //place operations which belong to all states here;
        //place operations deciding condition variables here;
        //place level-triggered blocks here;
        if(condition1){
            //operations
            ...
            RAM/ROM blocks(if any)
            function_calls(if any)
            goto state2_label;
        }
        if(condition2){
            //operations
            ...
            RAM/ROM blocks(if any)
            function_calls(if any)
            goto state3_label;
        }
    state2_label:
        ...
        ...
    end:
        return;
}
```

Major Challenges

Due to abstraction gap bewteen Verilog and C language, we faced some major issues are as follows:

1. Data inconsistency.
2. Signed conversion.
3. Data-width mismatch.
4. Level-triggered Operation.

1. Data inconsistency

- This issue occurs for RAW (Read after write) type of operations.
- When modified register values is used in other read in operation.

Example:

```
always @ (posedge ap_clk) begin
    if (1'b1 == ap_CS_fsm_state2) begin
    a <= b;
    c <= d;
    end
end
assign b = x + y;
assign d = a + m;
```

Fig. 2.6

```
ap_CS_fsm_state2:
    if (1 == ap_CS_fsm_state2)
    {
        a = x + y;
    }
    if (1 == ap_CS_fsm_state2)
    {
        c = a + m;
    }
```

Fig. 2.7 Generated C-code (Before)

```
ap_CS_fsm_state2:
    a_old = a;
    x_old = x;
    y_old = y;
    m_old = m;
    c_old = c;
    if (1 == ap_CS_fsm_state2)
    {
        a = x_old + y_old;
    }
    if (1 == ap_CS_fsm_state2)
    {
        c = a_old + m_old;
    }
```

Fig. 2.8 Generated C-code (After)

2. Signed conversion

- “Signed” keyword is used in Verilog code to convert the value into its signed representation in other words we must use its two’s complement value.
- Issue occurs if we use its +ve value in place of its original value.

For Example:

- In Verilog code:

```
assign X_reg = ( $signed(7'd117) + $signed(Y_reg) );      //(X=32 bits, Y=20bits}
```

- In generated C code (after):

```
X_reg = twos_complement(117, 7) + twos_complement(Y_reg, 20);
// X_reg = ( -11 ) +  Y_reg_new;     …….. (4)
```

3. Data-width mismatch

- In Verilog, if data-width of LHS and RHS of a operation is not same then it automatically adjust.
- If we directly realized it into C language than it can cause overflow or underflow issue during assignment.
- Solution is improve the rewriting method. As shown in Fig. 2.7

```
// RTL Code
        reg [20:0] r_A;
        reg [20:0] r_B;
        reg [31:0] r_C;
        wire [31:0] w_C;
        always @ (posedge ap_clk) begin
            r_C <= w_C;
        end
        assign w_C = r_A * r_B;
// Incorrect Generated C code
        unsigned long long int r_A, r_B, r_C;
        r_C = r_a * r_B;
// Resolved C code
        unsigned long long int r_A, r_B, r_C;
        r_C = (r_A * r_B) & 64d'4294967295;
```

Fig. 2.7

4. Level-triggered operation

- When “always(*)” is encountered in place of “always(pose edge)” then such block get triggered when the status of the registers on RHS in all the operations changes.
- Solution is to perform write before any read and use the updated value.

```
//RTL Code
 always(*) begin
  if(cur_state == 1'd2 & reg_X == 1'd1) begin
     b <= a;
  end
 end
 always(*) begin
  if(cur_state == 1'd2 & reg_X == 1'd1) begin
     a <= c;
  end
 end
 assign c = a + 5;
```

Fig. 2.8

```
//Incorrect C Code
 State_2:
  b_old = b;
  a_old = a;
  if(reg_X == 1) {
   b = a_old;
  }
  if(reg_X == 1) {
   a = a_old + 5;
  }
```

```
// Resolved C code
 State_2:
  b_old = b;
  a_old = a;
  if(reg_X == 1) {
   a = a_old + 5;
  }
  if(reg_X == 1) {
   b = a;
  }
```

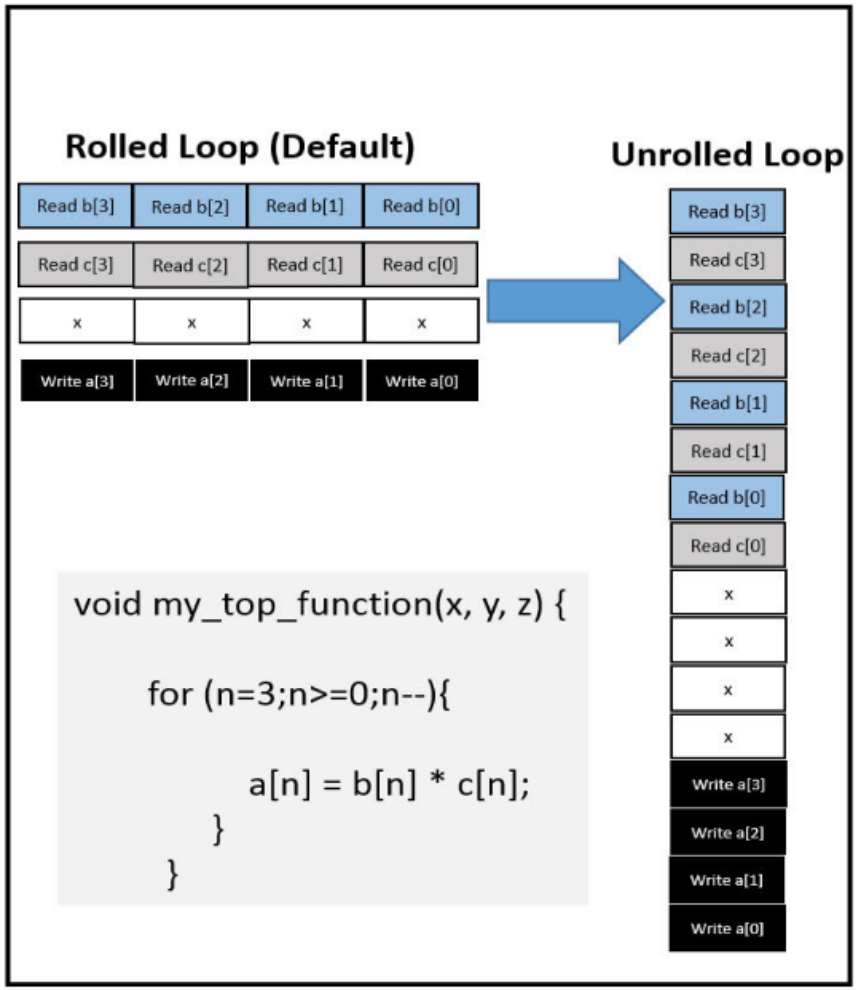
Fig. 2.9

Table of contents:

- Introduction
- Literature survey
- RTL to C conversion
- **Modelling Hardware Parallelism**
  - Loop unrolling
  - Instruction Level Pipelining
  - Task Level Pipelining
- Experiment Results
- Conclusion

Loop Unrolling

- Loop unrolling transforms the loops by creating multiple copies of the loop in RTL design, which allows some or all iteration occur in parallel.
- The loop unrolling increase the concurrent data access and improve the throughput and latency of the design.
- Loop unrolling can be done fully or partial unrolling.
- In both type the unrolled FSM model is similar to the baseline FSM where the register transfers form the iterations of the loop.
- Since the overall structure of FSM remains the same, our simulation model successfully emulates the correct functionality for a loop unrolled mode.



Instruction Level Pipelining

- It allows the concurrent execution at instruction level.
- In RTL, pipelined segment can be distributed across multiple states of FSM, each are referred as stages of the pipeline.
- Sample example of loop pipelining are shown in Fig. 3.0

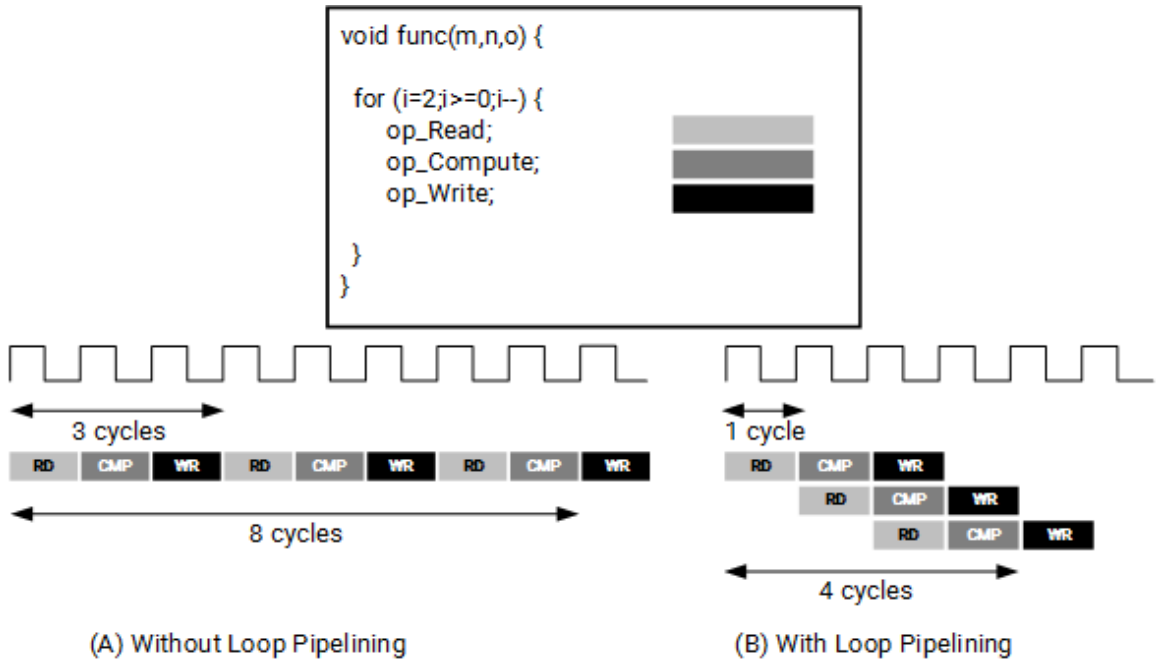


Fig. 3.0

Instruction Level Pipelining (Continued....)

- Each pipeline stages are divided into sub-stages which are usually controlled by some activation flag of that sub-stage.
- In RTL, we found that no two stage execute in parallel, but the sub-stages of stage can execute in parallel according to their respective activation flag.
- Instruction level pipelining can be applied on-
  - At Function : pipelined different instances
  - At loop: pipelined different iteration.
- Fig. 3.1 shows sample example function warp with loop pipelining.

```
void warp (int Img[ROWS][COLS], int Out[ROWS][COLS])
{
// a,b,c,d,e,f,i,j are defined as macros
 int x, y, destX, destY;
 loop-1.1:for(y = 0; y < 8; y += 1) {
   loop-1.2: for(x = 0; x < 8; x += 1) {
        #pragma HLS loop_flatten off
        #pragma HLS pipeline
        destX = (a*x + b*y + c*x*y + d);
        destY = (e*x + f*y + i*x*y + j);
        Out[destY][destX] = Img[y][x];
  }}
}
```

Fig. 3.1

Instruction Level Pipelining (Continued.....)

The representative C-code of pipeline stage are shown in Fig. 3.2

```
... // Code for other states.
STATE_S3: //pipelined state S3
 x_old = x; destX_old = destX; destY_old = destY_old; S3_1_flag_old =
     S3_1_flag; S3_2_flag_old = S3_2_flag;
 //Code to update S3_1_flag and S3_2_flag
 if(S3_1_flag_old == 1) { // S3-1 code block
   Img_CE = 1;
   Img_addr = addr_calc(x_old, x_old);
   destX = Calculate_DestX (x_old, y_old);
   destY = Calculate_DestY (x_old, y_old);
   x = x_old + 1;
 }
 if(S3_2_flag_old == 1) { // S3-2 code block
   if(x_old < 8){
      out_WE = 1;
      out_Addr = addr_calc(destX_old, destY_old);
   }
 }
 // other code in S3
```

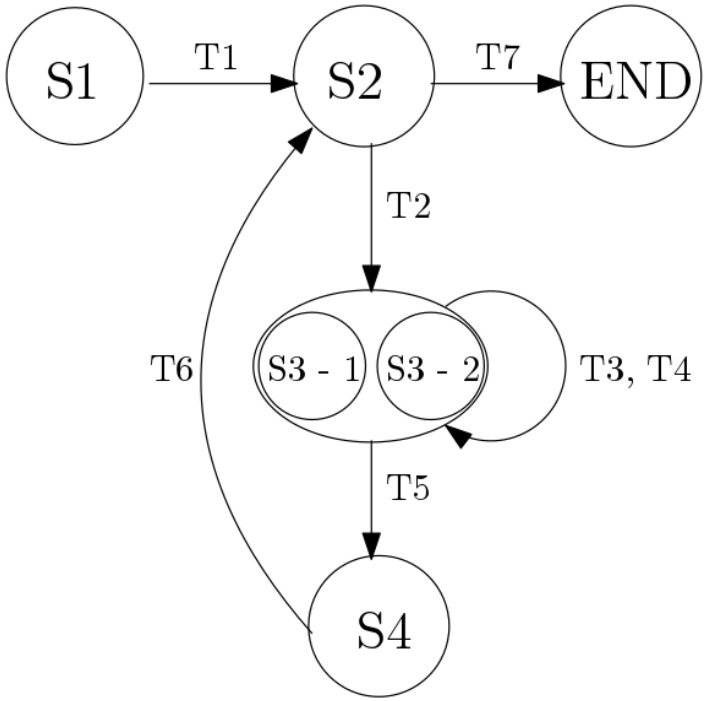


Fig. 3.2

Task level pipelining

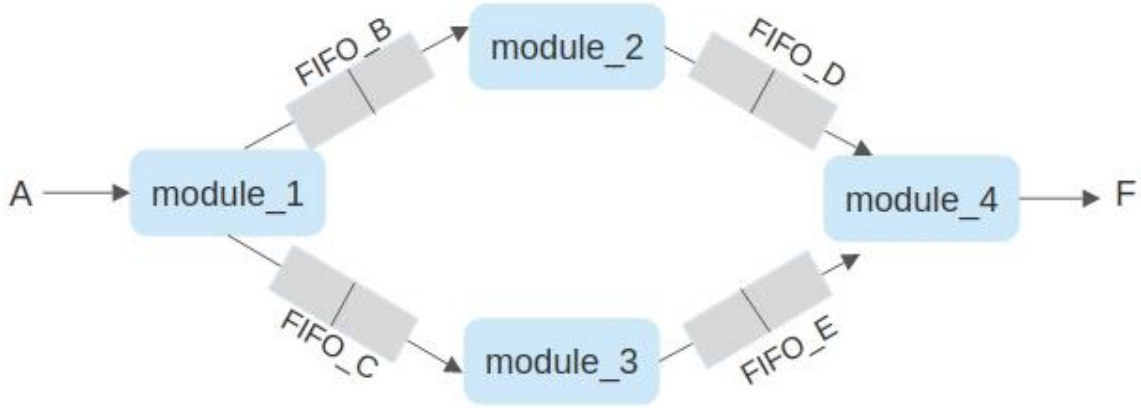
- FIFO Style and Ping-pong Style

```
void model_flow(int A[LIMIT], int
    F[LIMIT]) {
    int B[LIMIT], C[LIMIT],
        D[LIMIT], E[LIMIT];
    #pragma HLS dataflow
//HLS STREAM pragma is
//applied to B, C, D and E
        module_1(A, B, C);
        module_2(B, D);
        module_3(C, E);
        module_4(D, E, F);
}
void module_1(int A[LIMIT], int
    B[LIMIT], int C[LIMIT]) {
   int i;
    for(i=0; i< LIMIT; i++){
      B[i] = A[i] * 9;
      C[i] = A[i] * 2;
  }
  }
```

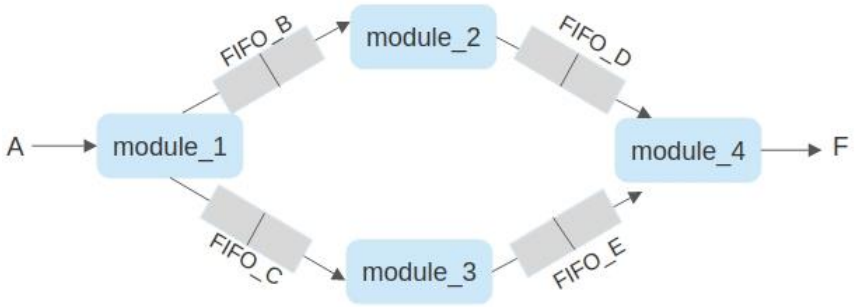
```
void module_2(int B[LIMIT], int
    D[LIMIT]) {
  int i;
   for(i=0; i< LIMIT; i++)
     D[i] = B[i] * B[i];
  }

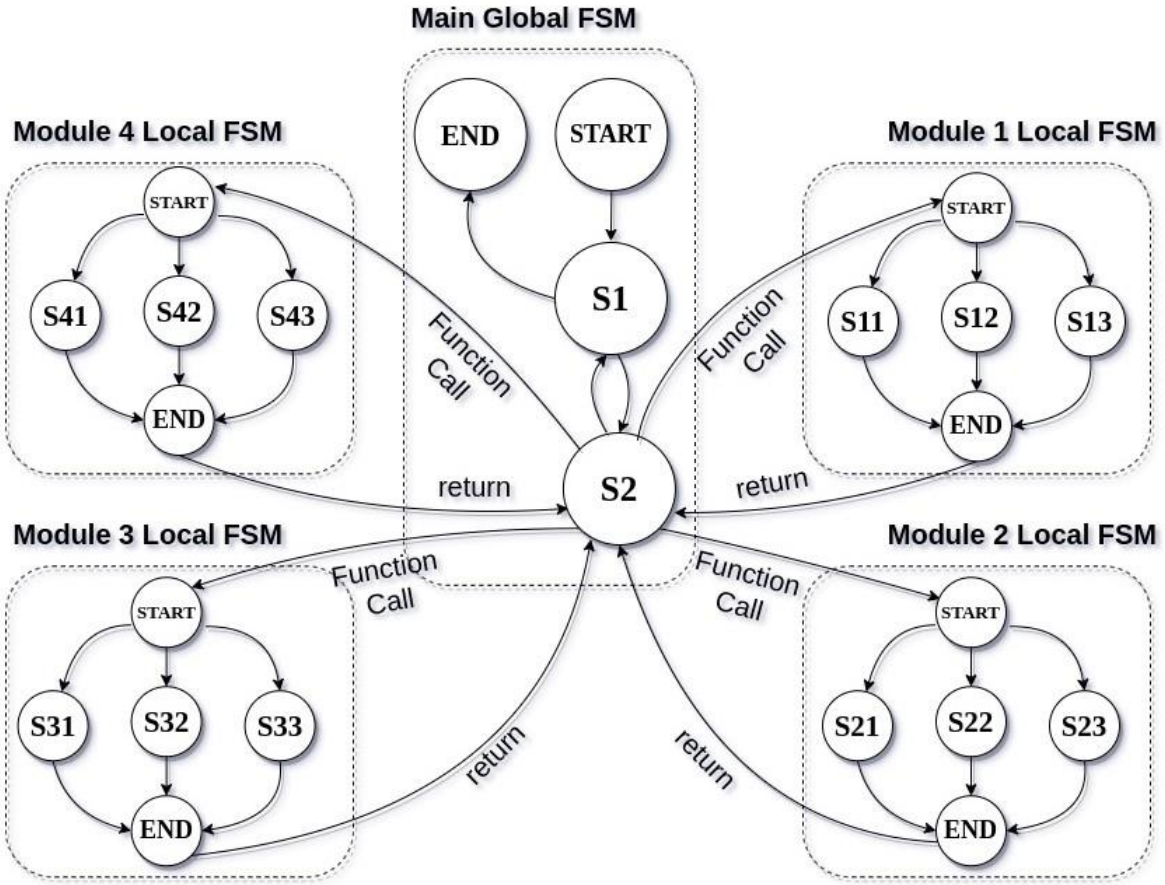
void module_3(int C[LIMIT], int
    E[LIMIT]) {
  int i;
   for(i=0; i< LIMIT; i++)
     E[i] = C[i] * C[i];
  }

void module_4(int D[LIMIT], int
    E[LIMIT], int F[LIMIT]) {
  int i;
   for(i=0; i< LIMIT; i++)
     F[i] = D[i] + E[i];
  }
```



Task level pipelining





Task level pipelining

Generated C-code of the top module of sample example.

```
state_1:
     if(ap_done) goto end;
     else goto state_2;
state_2:
     module_1(&m1_start, &m1_done, &is_empty_B, &is_empty_C...);
     module_2(&m2_start, &m2_done, &is_empty_B, &is_empty_C...);
     module_3(&m3_start, &m3_done, &is_empty_B &is_empty_C...);
     module_4(&m4_start, &m4_done, &is_empty_B &is_empty_C...);
     /* Following code segments ensure that a module starts only if the
         input FIFO isn't empty */
     m2_start = !is_empty_B;
     m3_start = !is_empty_C;
     m4_start = (!is_empty_E & !is_empty_D);
     ap_done = (m1_done & m2_done & m3_done & m4_done);
     goto state_1;
end:
```

Fig. 3.7

Task level pipelining

```
// All variables are declared static
 if(resume_state == 1) goto S21;
 else if(resume_state == 2) goto S22;
 else if(resume_state == 3) goto S23;
S21:
   if (m2_done == 1 || m2_start == 0) return;
   else  resume_state = 2; return;
S22:
   // Activate fifo_B handshakes for read
   fifo_B(ce_B, we_B, &done_B, &is_empty_B ....);
   resume_state = 3; return;
S23:
  if (is_full_D) { // check if fifo_D is full
   resume_state = 3; return;
  }
  // Code statements to Calculate D[i]
  // Activate fifo_D handshakes for write
  fifo_D(ce_D, we_D, &done_D, &is_empty_D ...);
  resume_state = 1; return;
```

Figure 21: Representative RTL-C code structure of *module_2*

Objective -- revisited

Our aim is to perform RTL to C reverse engineering to generate equivalent C-code.

MUST satisfy:

- fast simulation
- functional correctness of the RTL
- cycle accurate simulation
- accurate performance estimation
- generate a highly readable and debug friendly simulation code

MUST Supports:

- array mapping to memory modules,
- non-inlined function calls,
- parallel execution frameworks invoked by loop unrolling, pipelining, task level piplining etc.
- accurate simulation of pipeline stalls

Properties of the Benchmark

- We performed experiment on several standard example programs from CHStone Benchmark Suite and Bambu HLS Tool.
- *dfadd*, *dfmul* and *dfsub* are control intensive benchmark programs with several conditional statements but no arrays. Whereas larger benchmark programs like *des*, *aes* and *mips* are data intensive programs with several arrays and function calls.

Benchmark	#Line	#If-else	#Array	#Func	#Loop
aes_dec	949	24	15	13	13
aes_enc	979	24	15	11	13
des	354	1	9	6	11
mips	313	0	5	1	5
dfsub	955	34	1	17	0
dfadd	554	32	1	8	0
dfmul	522	28	1	17	0
arf	53	2	0	0	0
motion	52	0	0	0	0
waka	33	2	0	0	0

Parsing Result

Benchmark	#C	#RTL	#Gen-C	Parser Runtime(s)
aes_dec	949	3154	5776	0.334
aes_enc	979	2799	4784	0.237
des	354	2330	2856	0.189
mips	313	1779	5906	0.848
dfsub	955	2203	2856	1.097
dfadd	554	1724	2132	0.646
dfmul	522	2237	2858	0.593
arf	53	351	375	0.010
motion	52	415	471	0.014
waka	33	270	281	0.007

HLS simulation Result

- we compare our simulation Framework with Vivado HLS C-simulation, ModelSim RTL simulator, Verilator Simulator and Vivado HLS RTL Co-simulation (XSIM).

Benchmark	Simulation Time (seconds)								
	FastSim	C-sim	Speedup	RTL Cosim	Speedup	ModelSim	Speedup	Verilator	Speedup
aes_dec	20.176	14.442	0.72x	4467	221.4x	4780	236.9x	316.4	15.7x
aes_enc	19.32	12.656	0.66x	4389	227.2x	4693	242.8x	296.23	15.3x
des	34.43	28.01	0.82x	34672	1007.9x	36024	1047x	723.01	21.1x
mips	1.782	0.985	0.55x	2620	1455.5x	2885	1618.8x	20.4	11.33x
dfsub	0.807	0.717	0.89x	8	10x	14	17.5x	4.117	5.1x
dfadd	0.629	0.561	0.89x	7	11.20x	13	20.7x	3.92	6.2x
dfmul	1.062	1.374	1.29x	10	9.43x	17	16x	7.165	6.8x
arf	0.624	0.601	0.96x	6	9.7x	11	17.7x	3.93	6.33x
motion	0.491	0.565	1.15x	6	12.24x	10	20.4x	3.3025	6.73x
waka	0.411	0.454	1.10x	8	19.46x	11	26.77x	2.72	6.62x
Average			**0.91x**		**298.40x**		**326.45x**		**10.13x**

Results for Loop unrolling and Pipelining

- We also compare simulation time after applying loop unrolling and pipelining optimization.

Benchmark	FastSim(s)	Simulation Speedup		
		RTL Cosim	**ModelSim**	**Verilator**
aes_dec(u)	31.6	135.13x	142.47x	12.58x
aes_dec(p)	33.57	126.57	136.64x	12.77x
aes_enc(u)	26.07	155.96x	169.12x	14.48x
aes_enc(p)	32.62	127.49x	137.12x	12.58x
des (u)	41.4	697.05x	726.77x	17.4x
des (p)	46.5	797.22x	829.87x	20.35x
Average		**339.90x**	**356.99x**	**15.03x**

Results for Task level pipelining

- We apply dataflow optimization for both FIFO and PIPO style on some standard example and compare the simulation time with other simulators.

Benchmark	FastSim(s)	Simulation Speedup		
		RTL Cosim	**ModelSim**	**Verilator**
toy (pp)	23.7	20.46x	22.3x	6.06x
toy (ff)	26.6	18.73x	20.11x	5.4x
mergsort (pp)	31.2	25.2x	28.6x	8.7x
Insertionsort (ff)	39.5	21.7x	23.8x	7.6x
histogram (pp)	42	23.3x	26.2x	9.4x
FFT (pp)	153.2	117.6x	124.2x	10.1x
Average		**37.8x**	**40.9x**	**7.9x**

Performance Estimation

- We analyze and compare the performance of our framework as shown in Table on the benchmark examples.

Bench	Vivado Synthesis (Clock cycles)		FastSim (Clock cycles)		RTL Cosim (Clock cycles)	
	Min	Max	Min	Max	Min	Max
aes_dec	?	?	5654	5654	5654	5654
aes_enc	?	?	3006	3006	3006	3006
des	125065	125321	125425	125427	125425	1254257
mips	?	?	3383	3683	3383	3683
dfsub	8	21	9	19	9	19
dfadd	7	20	9	18	9	18
dfmul	8	22	12	20	12	20
arf	7	7	7	7	7	7
motion	6	6	6	6	6	6
waka	2	3	3	3	3	3

Summary

- we convert HLS generated RTL to equivalent C-code by taking advantage of RTL structure.
- The generated C-code is used for faster simulation which is around 300 times faster than tradition RTL simulators.
- Our tool also support advance HLS optimization such as unroll, pipeline and dataflow.
- fast simulation
- functional correctness of the RTL
- cycle accurate simulation
- accurate performance estimation
- generate a highly readable and debug friendly simulation code

Thank you